In this chapter, you'll see:

- A tour of the included matchers
- The difference between *primitive* and *higher-order* matchers
- The difference between *value* and *block* expectations

CHAPTER 11

Matchers Included in RSpec Expectations

In the previous chapter, you learned how to write expectations to check your code's behavior. You got to know the various parts of an expectation, such as the *subject* and the *matcher*.

Now, it's time to take a closer look at matchers. You've called them in your specs and combined them with other matchers. You've even written a simple one from scratch, though most of the time you won't have to. RSpec ships with a ton of useful matchers to help you specify exactly how you want your code to behave.

In this chapter, we're going to take a tour of RSpec's built-in matchers. We won't exhaustively list every available matcher—that's what Appendix 3, *Matcher Cheat Sheet*, on page 307 is for. But we will hit the highlights so that you can choose the best matcher for each situation.

The matchers in rspec-expectations fall into three broad categories:

- *Primitive matchers* for basic data types like strings, numbers, and so on
- *Higher-order matchers* that can take other matchers as input, then (among other uses) apply them across collections
- *Block matchers* for checking properties of code, including blocks, exceptions, and side effects

We'll start our tour with the most commonly used primitive matchers.

Primitive Matchers

The word *primitive* in a programming language refers to a bread-and-butter data type that can't be broken down into smaller pieces. Booleans, integers, and floating-point numbers are all primitives.